

AI Agent Memory Infrastructure 系统设计手稿

RAM 端与外围 · 面试深度版（推演汇总 · 不变量升维 · 口述骨架 · 机制预研 · 追问弹药库）

2026-07-11

0. 题设与全局锚定

目标岗位：ByteDance AI Agent Memory Infrastructure, Senior SWE。JD 核心职责：memory 全生命周期 —— ingestion、storage、indexing、retrieval、updating、compression、forgetting；难点清单：memory representation、retrieval & ranking、conflict resolution、summarization & fusion、lifecycle management、multimodal fusion。

Master 锚（全篇唯一，所有数字由它派生）：

量	数值	推导
用户规模	1B DAU	题设
消息量	240 条/人/天	题设
单条大小	2KB raw	175PB/年 反推
日消息总量	480 TB/天 raw	$1B \times 240 \times 2KB$
年消息总量	175 PB/年 raw	$\times 365$
消息 QPS	2.8M/s 均值, ~8M/s 峰值	$1B \times 240 / 86400$, 峰值系数 $\sim 3 \times$
并发 session	~1.67 亿均值	Little's Law: $1B \times 4h / 24h$ (人均在线 4h = 240 条 $\div$ 1 条/分钟)

锚定纪律（本次推演反复验证的方法论）：任何新估算出来必须先与 master 锚对撞，撞上才算数；改锚是全局事件，所有下游数字同缩，新旧不得混用。本次推演中两次账目滑坡（"200 万并发"侧估、0.5KB/条口径漂移）都是靠这条纪律抓回来的。

1. 存储分层全景（上一场推演的结论）

1.1 三层介质台账

层	存什么	窗口	压缩	落盘	访问模式
热 RAM	活跃 session state	活跃期 + TTL 尾巴	2-3:1 (record 级)	稳态 20-50TB	点查/range, 读带宽绑定
温 SSD	近 3 个月原文	~90 天	5:1 (ORC 批压缩)	~8.8 PB	cache miss 回载、近期翻查、增量炼化输入
冷 HDD	全量历史	1 年+/retention	5:1 + EC 1.4 $\times$	~49 PB/年	纯批：重放、重炼、合规

关键裁决：

- 压缩比是分层属性，不是全局常数：**大 block 批压缩换高压缩比（数仓 5:1~8:1），record 级小块压缩换随机访问（热层 2-3:1）。对话语料（LLM answer 模板化、跨 session 重复多）自训字典可达 8:1，若改锚则温层 5.5PB、冷层 ~31PB 全局同缩。
- 副本系数按层配：**3 $\times$ replication 只给热/温 serving 层（读吞吐 + 快速恢复）；冷层用 erasure coding (RS 10+4, ~1.4 $\times$)，成本减半。冷层便宜，不是题眼，被问成本时一句话甩出，不主动展开。
- RAM 层不 bulk-load：**serving 是 retrieve top-k（几 KB~MB working set + 热索引进内存），不是把温层扫进内存。8.8PB 进不了内存，也不需要。

1.2 双路 + 回灌（架构脊梁）

conversation 不是"存在某一层"，是同一份数据走两条路、喂两个系统：

1. **在线路（TAO 式，低延迟）**：消息双写之一 → 在线 KV/graph（毫秒级），服务显示 + 推理时 recent context；
2. **离线路（DataSwarm/Hive 式，数仓）**：同一份消息 log 进数仓（全量 ORC），Presto/Spark 离线挖掘 → 炼 embedding/summary/profile；
3. **回灌闭环**：炼出的 refined memory push 回在线服务层（向量索引/KV），供推理时低延迟 top-k 检索。

面试主动讲这条 spine；每条路内部（TAO shard 亲和、ORC 编码、Presto MPP、回灌 pipeline）兜里揣着，被问再挖。**本场推演把 TAO 的 UI 显示职责划出界，只保留"在线 KV 存近期"这个角色**——inference 相关是本组的边界。

2. Inference 侧到底存什么：可重算三分法

本场第一条主线。判据一个不变量：

可重算且便宜 → 不存；可重算但贵且高频复用 → 存（cache/index）；不可重算 → 必须存（log）。

2.1 不存 QKV/activation（两个硬理由）

1. **信息量增量为零（本质理由）**：最后一层 hidden state 是 $f(\text{raw text}, \text{model weights})$ 的确定性函数，相对 raw text 边际信息量为零——它不是数据，是缓存的计算。且当训练数据用也省不掉重算：backprop 需要全层 activation 在 forward 时重建。
2. **量级过不去**：KV cache ~320KB/token（70B 级： $2 \times 80 \text{ 层} \times 8 \text{ KV head} \times 128 \text{ dim} \times 2\text{B}$ ），是原文（~4B/token）的 10^5 倍。 $1.2 \times 10^{13} \text{ token/天} \times 320\text{KB} \approx 4 \text{ EB/天}$ ；只存最后一层也 ~380 PB/天。且 KV 是 model-bound——模型一升级全部作废。

合理内核的正经形态：蒸馏存 top-k logits（~100B/token，对 student 有真信息量，采样存不全量）；最后一层的语义已经以 embedding（pooling 后 1~4KB/chunk）的形式在清单里。

2.2 存的清单（收敛后）

1. **Raw text**：source of truth, model-agnostic, 任何模型换代从这重炼——第三类（不可重算的原始输入）；
2. **Inference-time context 快照 log**：当时实际进 context 的完整内容（检索结果、system prompt、tool 输出、profile 快照）+ model version + 采样参数。**时变系统状态的函数，事后重放不出来**——这是训练 memory 检索/排序模型的核心数据，等价于 ads 的特征快照（feature logging），防 training-serving inconsistency；
3. **Embedding（划归检索块）**：可重算但便宜（~1PB）、全局高频复用、且 ANN/检索在结构上要求索引先于查询存在——身份是**物化的索引**（同倒排：能重建但没人存）。带 embedding model version tag，升级要全量 reindex；
4. **不存**：QKV（第一类的反例：可重算 + 贵 10^5 倍 + 复用局限单 session）；model 本身（registry 指针）。

KV cache 的正确定位：推理引擎侧的分层 serving cache（HBM→CPU RAM→NVMe），分钟~小时级 TTL、session/prefix 粒度，不进数仓。

注意 embedding 生产的架构切分：推理 LLM 的 hidden state pooling 做检索 embedding 质量差（causal LM 目标 ≠ 相似度目标，各向异性）；生产用独立的对比学习 embedding model（双塔/EBR）。两个模型两条路，接口是 text，各自独立演化。

3. 检索版图：内外两半与信号生产线

3.1 版图切分

- **内部 memory（自有 corpus）**：全栈可控——建索引、hybrid、训双塔、reindex；
- **外部搜索（黑盒 API）**：没有索引权，控制点只剩两端——进端 query rewriting（对话上下文 → search-friendly query），出端 rerank（返回的 top-50 现场 encode 重排，ANN 的结构约束只废掉检索、废不掉重排）。

统一的层次：**接口层**统一（两路皆 retrieve(query) → passages + source + score 的 tool）；**融合层**统一（cross-encoder 汇点，两路候选同台，分数才可比）；**信号**统一（utilization log 不分内外，一份日志喂 query gen、双塔、router、reranker 四个模块）；**索引所有权**不统一（物理硬约束）。

Query 侧："一次理解，两次渲染"——意图抽取共享（指代消解、实体补全），渲染分叉：内部头保留用户私人词汇（alias 表个性化灌这支），外部头 web 术语化 + 实体全称 + 时间限定。**统一在语义，分叉在词表，且只有一支词表是私人的。**

3.2 v0 裁决：lexical + LLM query rewriting

- **理由一（场景特有）**：个人 memory 检索里 query 和 corpus 出自同一个人的词概率分布——用户找自己说过的话，用词高度一致，vocabulary mismatch 天然弱，lexical 天然强；
- **理由二（benchmark 实锤）**：BEIR 的核心发现——BM25 zero-shot 域外碾压大批 dense；dense 只在有域内训练数据时稳赢；
- **理由三（ROI）**：零训练依赖、一周上线、无 pseudo-query 噪音。

Dense 的进入条件：规模化相关性信号。而这个场景没有天然 click，三重先天缺陷：

1. **没有 SERP**：用户看不见"召回了哪些 memory"，不存在用户动作意义的 click，utilization 是模型侧推断的 label；
2. **corpus 是 per-user 的**：信号 pool 不到 doc 级（doc-ID embedding、popularity prior、query-doc 共现全没了），只能在属性/内容空间学、跨用户迁移——Google 的 personal search（Gmail/Drive）文献同一困境同一解法（属性级参数化）；
3. **量本身**：每轮才一条推断 label。

由此**双塔若训，doc 塔无 memory-ID 参数，纯内容编码 + 属性特征**——被逼的约束反而利于迁移。这也反向加固 v0：连模型结构都被逼到内容空间，lexical 的相对劣势更小。

3.3 信号生产线（五级 + session 层）

从便宜到贵、从弱到强的级联，不是选哪级而是级联——曝光→点击→转化→人工评测的推荐老配方换皮：

级	信号	来源与成本
0	自监督伪任务（ICT 对话版：第 N 轮当答案，反推 query）	免费，冷启动，分布 gap 已知
1	执行反馈（召回空/分数分布/agent 重试=免费负标注器）	系统自产，上线即有，量最大，当主食
2	Utilization（answer 是否依赖该 chunk，LLM 判/引用标记）	采样精标，当 finetune
3	用户结果信号（"我不是说过吗"=强负例；regenerate/顺利推进）	免费但稀疏延迟含噪，归因难（信用分配）
3.5	Session implicit feedback ：下一个 prompt 是对上一个 query 的评分——reformulation=负、深入追问=正、换话题=无标注。密集（每轮都有）但必须用下层信号消歧归因（query 烂/库里没有/生成没用好，三个责任方）。 用作 reward 不用作 label （下一 prompt 已被失败污染，分布偏移）	免费，密度高一个量级；web search 二十年 session 分析（query reformulation as implicit feedback）平移
4	金标集（人标 + ablation 反事实）	最贵，只做评测和上面四级的校准器

配套病理学原样平移：只见被召回的 → bandit feedback/IPS；LLM 偏用排前的 chunk → position bias/PAL；采纳信号延迟含噪 → delayed feedback。

Query generation 的训练阶梯：v0 prompt（history 在 context 里 = 无监督白拿的个性化先验）→ v1 rejection sampling（只留检索成功的）→ v2 RL（reward = 可验证检索指标，Search-R1/DeepRetrieval 路线，query 分布是策略自己探索的真实分布，绕开 pseudo-query gap）。**表征无监督白拿，策略必须付费，费用就是 utilization log**——LM 目标 ≠ query 效用目标（概率高的词恰恰不判别，TF-IDF 存在的原因）。

Agentic search vs embedding retrieval 的分界账：agentic 花 token 省索引（Claude Code 纯 grep 迭代），embedding 花索引省 token（Cursor 建索引）。低 QPS 高价值可迭代 → agentic 赢；1B 用户每轮检索、latency 在 critical path → query gen 是检索层前端而非替代，账在 QPS 不在效果。

先发优势的精确表述：Gmail 搜索有真 SERP click——per-user corpus 上二十年真实点击，模型（属性级相关性、query 理解）warm-start 可迁移；但资产在模型和方法论里，不在 raw content 里（合规墙：purpose limitation）。对没有这份遗产的玩家（字节），signal building 不是可选项，是护城河的建造现场——这正是该组存在的理由，也是 logged feedback 背景的直接映射。

面试封坛段：检索层 v0 用 lexical + LLM query rewriting——同人词汇分布、零训练依赖、ROI 最高。dense 我知道怎么训（双塔/in-batch negatives + logQ/hard negatives 课程/IPS 去偏/Margin-MSE 蒸馏），但前提是规模化相关性信号，而这个场景没有天然 click——没有 SERP、corpus per-user、label 靠推断——所以 dense 不是 day-1 组件，是 signal building 做成之后的 v1。信号怎么造，本身是这个系统最难的子问题。

4. 业界对齐（mem0 基准，JD 点名）

骨架六步全部站住，三处偏差修正（都在写侧）：

对上的：读路径 = recent messages + summary + 检索 top-k 拼 context → 生成；检索是 hybrid（语义向量 + BM25 词形还原 + 实体匹配 + 时间意图打分，rank fusion 出 top-K）——lexical 在场，v0 结论被原样验证；写回异步不挡生成。

三处偏差：

1. 炼化触发是 **per-turn 增量**，不是 **session 结束批**：mem0 每个消息对到达即触发 extraction（用 summary + 近期窗口做上下文），summary 由独立异步模块周期刷新。正确形态是双轨：每轮异步抽 fact（主路径）+ 周期 summary（辅助轨）；
2. 冲突消解从写时漂移到读时：论文版写时 LLM 判 ADD/UPDATE/DELETE/NOOP；生产版改 **ADD-only**——新旧 fact 并存 + 时间元数据 + 矛盾标记，写时只做哈希去重，矛盾留到检索时 temporal reasoning 裁。钝赢精巧；
3. **task 聚类取 context 是超纲设计**：业界标准 = 最近窗口 + summary + 检索补充，per-task 线程化当亮点提、不当默认架构讲。

5. E2E Workflow：T0–T6 总览

一个 prompt 的完整旅程，六跳与 JD 职责的映射：

跳	动作	JD 对应
T0	读 session state（recent turns + summary），本机内存	storage / indexing 消费端
T1	routing + query 生成（读 profile/alias 点查）	retrieval 前端
T2	检索执行：本机倒排 + 向量内积 / 外部 API → rerank 汇点	retrieval & ranking
T3	context assembly：token budget 内组装四路	context engineering（JD 点名）
T4	调推理（RPC，另组），context 过网	—— 接口
T5	异步写回四件：append state、双写数仓、context 快照 log、fact 抽取	ingestion + storage
T6	周期批处理：炼化 + compaction	updating / summarization / conflict / compression / forgetting / lifecycle

Critical path 上只读不写（写全部推迟到 T5/T6 异步），这是整条 workflow 的第一设计原则。T6/T5 是本组心脏（区别于纯存储组的那部分），T0–T4 一半是通用件。

外围接口清单（核心模块看到的外部世界）：ingest(conversation_delta)（at-least-once，乱序有界）+ T+1 数仓可重放；retrieve(user_id, query_context, budget) → ranked memories（带 provenance）+ get_profile；租的服务 = 存储引擎 + GPU inference（RPC + quota）+ 外搜/routing/生成质量（邻组）；贯穿旁路 = context 快照 log + forget(user_id, scope) GDPR 级联删除口（day-1 约束所有存储设计）。

6. T0 深挖：Session State（本次推演最重的一跳）

6.1 存储选型与 schema

- **有序 KV（宽列），LSM 引擎**：字节语境 Abase/Bytable，开源对标 ScyllaDB/Cassandra（Discord 万亿消息同构）、Meta 系 MyRocks/ZippyDB。LSM 红利：写 append 友好、最新数据天然在 memtable/高层（“最近对话”读模式白吃）、TTL 原生（compaction 时丢过期）。
- **Key = (userid, sessionid[, time_bucket])**，时间降序，clustering key 排序白送 range scan。**taskid 不进 key**（查询时 taskid 尚未算出——分类依赖读出的 context，进 key 就循环依赖）；正确读法：有界读（最近 N 条/T 分钟/内存上限三取一）+ 应用层按 task 过滤（N 百量级，app-side filter 零成本）。time_bucket 学 Discord：防超长会话打爆单分区。
- **持久性档位**：写内存副本（3 replica ack）+ 异步 append log 落盘——副本冗余而非同步刷盘，丢失窗口毫秒级、最多最后一条（用户重发即可）。session state 非账务数据，一致性要求“还好”。
- **Task 聚类的系统契约**（模型本身归算法组，本组定限制）：query embedding 对 K 个 session 内 centroid 内积（K 个位数，微秒级）；零信号短 prompt（“那第二段呢？”）→ **sticky to thread**（置信度低于阈值沿用上一轮 taskid，对话连续性先验，便宜且准）；新类 = max 相似度低于阈值 O(1) 开。

6.2 容量：Little's Law 三用之一（字节存量）

单 session state $\approx 240 \text{ 条} \times 2\text{KB} = 480\text{KB raw}$ ，压缩 $\sim 110\text{KB}$ （含 centroids + 元数据）。

核心澄清（本次推演的一个卡点）：480TB/天是**流量**，不是**存量**。热池是直通缓冲区不是蒸馏器——不筛选、不提取，只管驻留；每条数据只泡 session 生命周期（ $\sim 4\text{-}5\text{h}$ 含 TTL 尾巴）：

存量 = 流入速率 $\times$ 驻留时长 = $(480\text{TB}/86400\text{s}) \times \sim 16000\text{s} \approx 90\text{TB raw} \rightarrow$ 压 2-3:1 $\rightarrow$ **30-45TB**

与并发口径（ $1.67 \text{ 亿} \times 110\text{KB} \approx 18\text{TB}$ ）同量级咬合。"全天数据都要随机访问" = 160TB 是反事实上界（它的意义：随机访问版 vs 批处理版 60TB 的 $2.7\times$ 差价 = 访问模式决定压缩比的价格标签），实际台账只有热 50TB + 数仓 60TB/天两个数。

6.3 逐出：TTL/LRU 钝件的胜利（业界对齐后的版本）

业界实证：Discord/Messenger 没有 session 感知的 RAM 池——持久 LSM 当主存（SSD 点查毫秒级够 serving），前面通用 TTL cache，逐出交给 LRU；AWS 参考架构 = Redis(TTL) + DynamoDB + 批 summarization。"炼化完成才逐出"这个精巧条件没人写——炼化输入走数仓/log 那路，不依赖 RAM 里那份，两条路本来解耦。

Work solution（封坛版）：

50TB quota + 2h 滑动 TTL（从最后活动起算——chat 负载每条消息即一次写、写时重设过期，固定与滑动自动等价） + jitter + volatile-lru + 持久层 miss 回滚兜底。

四件各司其职：TTL 管常规逐出、LRU 管压力分配、quota 管护栏、回滚管正确性。TTL 数值 2h 正中业界主流档（教程/参考实现 30min-2h）。TTL 选择本质是回访问隔分布问题（重尾、CDF 拐点），业界偷懒更优：TTL 设上界 + volatile-lru 在线逼近——**LRU 本身就是对回访概率的在线非参估计**。

6.4 Miss 率：分母检查与突发-内存共轭

分母纪律：命中率 per access，不是 per segment（多 AI 对抗中 GPT 表格的错误：只数段首、扔掉段内几十上百次命中，结论差一个量级）。

结构：滑动 TTL 下段内（间隔 1min $\ll$ 2h）永不过期，miss 只发生在间隔 $> 2\text{h}$ 的段首。碎片化模型（一天 4h 切 k 段）：段越碎间隔越短、越被 TTL 盖住——**miss 率对碎片化钝感，对长间隔次数敏感**，而长间隔由生活节律锁死（隔夜 + 日内 2-3 个深谷 $\approx 3\text{-}4$ 次/天）：

miss 率 $\approx 3.5/240 \approx 1.5\%$ （区间 0.4%-2.5%；极端碎片上界 20 段/240 = 8%，且题设 240 条两两隔 2h 需 480h $>$ 一天，" $\sim 0\%$ 命中"物理不可能）

Miss 代价：40K miss/s $\times$ 100KB 回滚 $\approx 4\text{GB/s}$ 打持久层，零头；单次 penalty 几 ms 落段首，用户无感。**容错性极好，miss 率高一档系统无感。**

突发-内存共轭（这一跳最深的一句）：均匀到达（每 6 分钟一条 $\ll$ 2h）= 零 miss，但滑动 TTL 让全员常驻 $\rightarrow$ 110TB 爆 quota；真实的突发世界里深谷主动请退非活跃用户 $\rightarrow$ 20TB。**miss 和内存是共轭量，用户的睡觉时间就是我们的内存预算——那 3-4 次段首 miss 是内存省 5 倍的收据。**TTL 是在共轭对之间划刀的位置。

三重闭合的容量论证：均值账（Little's Law 20TB $<$ 50TB）、分布账（共轭保证不滑向全驻留）、排序账（80/20 重尾：驻留按人头、命中按流量，LRU 天然把内存花在流量大户上，流量加权命中率 $> 99\%$ ）。

严格 vs 工程：严格解是滑动 TTL 下的 clump/busy-period 过程——驻留时长是间隔分布的泛函（ $P(\text{gap} > \tau)$ 定簇数、截断均值定簇长，同样日均 4h，均匀撒 vs 三大块驻留差 4 倍），重尾下解析难。工程解：**trace-driven simulation**——30 天真实访问日志离线重放 TTL+LRU 逻辑，稳态/miss/峰值/分位数/参数扫描（ τ 从 1h 到 8h 的 miss-内存曲线）全部直接读出，挂成周期回归作业，行为漂移先于生产报警。**解析证结构，仿真出数字。**

6.5 架构：计算随数据（a）vs 无状态 serving（c）

物理层判据：读数据的 CPU 和存数据的 RAM 是否同机。同机 = 内存总线（ $\sim 100\text{GB/s}$ ，100ns），NIC 无关；跨机 = 两边 NIC 各过一遍（ $25\text{GbE} \approx 3\text{GB/s}$ ，100 μs ），慢 ~ 30 倍。

两个正交决策：请求按 userid 路由（一行配置，便宜，谁该做） $\neq$ serving 进程与 KV 分片同机部署（网络税的真正开关）。

读带宽账 ("读 heavy 指字节量")：读 ~100KB context vs 写 ~2KB，差 50 倍；2.8M 读/s × 100KB ≈ **300GB/s 聚合**，摊 80 台 KV 机每台 3.5GB/s——25GbE 打穿。破法按 ROI：① delta read (serving 侧 session 级软缓存 + 游标增量读，段内 2-4KB，300 → 12GB/s，25 倍)——它是 cache 不是副本 (丢了全量读重建，本来 T3 组装就要持有这份内存，只是"别扔")；② 传压缩块；③ 加机器摊薄 (为网卡买内存机，笨)。

方案 a (计算随数据)：前端路由层按 userid 一次 fanout → 数据所在刀片本地组装，读走内存总线，读税归零。**4000 台口径下成立且更强**：2PB 热池 ÷ 500GB 可用 = 4000 台；50TB session state 摊每台 13GB (2.6%，寄生)；1TB 机型 = 250GB 分片 + 500GB 计算 + 250GB 系统，比例健康；per-user 全家 (state、倒排、向量、fact) 按 userid 共置同一分片，T0-T3 全程本地。代价台账：编排发版滚动重启带状态节点 (先移分片责任)；serving 弹性与分片绑定 (re-sharding 节奏锁死——KV 分片长期稳定 vs serving 分钟级弹性)；热点疏散弱 (补丁：virtual shard 切细、分钟级迁移)。

方案 c (业界默认)：无状态 serving + 远程 KV + delta read，12GB/s 小税换两层独立伸缩 ("计算无状态、状态下沉"铁律)。

裁决：两个山头都是合法解，选型看组织 (存储与编排同团队 → a 可忍；跨团队 → c 的接口解耦值钱)。面试默认报 c，a 当"延迟预算极紧时"的口袋方案。**路由亲和要，部署亲和看情况。**

Context → GPU 那条腿谁都不省不掉：2.8M × 300KB (完整组装 context) ≈ 840GB/s，是 workload 固有流量，架构选择只决定在哪条腿上额外交税。GPU 集群侧倒推百万 GPU 量级 (2.8M QPS × ~1 GPU-秒/turn)，840GB/s 摊几万台推理机每台几十 MB/s——**网络不是难题，那百万 GPU 本身才是**。真正的砍刀：**prefix caching**——session 亲和路由 + 增量可辨识 (seqno)，公共前缀 KV 已在显存，每轮只传增量 ~10KB，840 → ~30GB/s。**Delta read 在推理侧的镜像，同一思想在两层各长一次**；跨组接口标准表述："我们保证 session 亲和路由 + 增量可辨识，推理侧吃 prefix cache 红利。"

7. T1-T4 读路径

7.1 T1：决策的消灭与摊销 (业界对齐后)

独立入控制器 + 价值预估器 = 相对业界的过度设计。业界两层：

- **热层 (profile + 高置信 fact + recent turns)**：无条件每轮注入，零决策——几百 token，判断成本 ≥ 注入成本，决策不如不决策 (ChatGPT memory 即此形态)。**决策被消灭**；
- **冷层 (历史对话检索/外部搜索)**：暴露成 tool，生成 LLM 自己调——判断搭生成的便车，边际成本零 ("agent 决定什么时候查")。**决策被摊销**。"2.8M/s 判断比生成还贵"是伪问题：不存在独立判断调用。

Pre-router 的正确身份：10ms 小模型/规则不是决策者，是 **prefetch 触发器**——高置信时提前并行发检索，把 3-5s 外搜藏进延迟；权威决策仍在 tool call；猜错代价可恢复 (白搜一次/agent 补调)。Bing Chat 的 search-intent 分类器是实锤。**Router 管快，tool call 管对**。学术 gating 线 (FLARE/Self-RAG/prefix-logits 嫡门控) = 口袋知识非默认架构。

Pacing 框架内残留：分层按成本排 (免费常注入、贵的按需调)；预算控制退化为粗粒度 quota/rate limit。界外删除的决策变量：生成用哪个 model (model/app 组)、全局 GPU 池分配 (平台组)。

7.2 Fact 层

- **形态**：结构化明文条目 ("用户住 Jersey City") + 时间元数据 + embedding 索引 (明文是本体，embedding 只是索引)。来源 = T5 逐轮异步 LLM extraction + 用户显式"记住 X"。
- **量**：~50-200B 明文 + 1KB embedding (1024 维 int8；5000 维超纲——生产主流 384-1536，per-user 小库撑不起，样本数 < 维度)；设计口径 300KB/人/30 天 → 全量 **300TB**，热部 (活跃用户随 session 加载) ~**50-100TB**，版本 buffer (多 embedding 并存：升级双版本 2×、多任务多空间、A/B) → quota **200-300TB**。
- **遗忘 ≠ TTL (本组本体论)**："对花生过敏"十年不访问不能过期。**TTL 管缓存，取代 + 衰减管记忆**：被取代 (新 fact 标记旧的，ADD-only + 时间裁决)、衰减降权 (不删只沉底)、容量逐出 (per-user 几百~几千条 quota，撞顶触发**压缩**——碎 fact 合并 summary——而非删除；quota 是"该炼化了"的信号不是硬阉)。

7.3 T2：检索执行

- **Per-user corpus 不需要 ANN**：240 条/天 × 年 ≈ 10⁴-10⁵ 向量，brute-force 内积 CPU 毫秒级——精确内积就是终解，HNSW 是全局十亿级 corpus 的工具 (用户 prompt 搜全局共享库——视频/post/好友——才复活，那是推荐检索的地盘，别的组)。
- **检索序**：本 session 优先 → 跨 session KNN 串行 + 时间衰减；内部 per-user 倒排 (离线批建 + 增量 append + 定期 compact，成熟数据活) + 向量 hybrid；外部 = API。
- **Rerank 标准**：该知识对回答此问题有无 clue，裁判是下游 LLM (utilization)；v0 LLM 直判，v1 蒸小模型。

7.4 T3/T4：组装与传输

- **In-flight 账（Little's Law 三用之三）**：2.8M/s × 响应时长。60s cap → 1.68 亿 in-flight × 300KB = 50TB cap；均值 ~10s → ~8TB 典型。报法：预算按 cap，预期按均值。
- **两档 SLO**（业界超对齐：搜索 API P50 0.45-2s，工具超时激进 3-5s，交互 TTFT < 500ms，单 turn 220ms-7.5s，多工具 10-18s）：**交互档**（默认）= 搜索 timeout 3-5s、最多 1 轮外搜、整 turn < 10s；**深度档**（agentic）= 60s / 单搜 10s / 最多 3 轮。档位由 tool call 顺手定。
- **显存以 token budget 形式进入本组设计**：context 长度 = 对方 KV cache 显存消耗（并发 × 长度 × per-token KV 定推理卡数），T3 预算从推理侧显存/成本倒推——压缩/炼化的存在理由有一半在此（用最少 token 传递最多记忆）。
- T3 输出 = system prompt + recent turns + 检索结果 + profile 摘要，预算切分 + provenance 排序；消耗上报（本次吃掉多少 token 预算、还剩多少），裁量权在上游——**我们是计量表，不是收费站**。

8. T5/T6 写侧（本组心脏）

8.1 T5：per-turn 异步四件写

生成完成后异步（critical path 零写入），十分钟级延迟可容忍——新鲜度不塌的原因：**同 session 的"刚说的话"由 recent turns 兜着**（本来在 context），fact 库晚十分钟只影响跨 session 检索，而下个 session 最早几十分钟后——异步窗口塞进天然缝隙。

1. 本轮 (prompt, answer) append 进 session state（2KB/次，QPS 成熟 KV 无压力）；
2. 双写落数仓（raw text 真身）；
3. **Context 快照 log**（append 型，含可能的 embedding feature）；
4. Fact 抽取（远程异步调用，见 8.3 成本结构）。

发起方在 session 刀片、直连 RPC（跳过 API 网关层，省一跳排队/序列化）；**GPU 计算不搬过来**——GPU 效率靠跨用户 batching，天然集中："CPU 计算随数据，GPU 计算随批次。"

8.2 T6：两条腿（T5 管快进，T6 管不烂）

结构类比：T5/T6 = LSM 的 memtable append / compaction。 Append 逃逸写延迟，把一致性和精简欠成债；T6 还债。

腿一（炼化产出）：原子 fact 聚合成 user profile/习惯/alias 表（跨轮跨 session 的模式，单轮视野看不见森林——profile 不是 T5 能完成的，T5 抽原子、T6 炼分子）、embedding 重算、session→日 summary 金字塔（summary 是 T5 抽取的输入，自耗环，必须独立周期跑）。= 推荐系统的 user-side feature pipeline，对象从 click 流换成对话流。

腿二（compaction，库的 GC）——JD 的 compression/forgetting/conflict resolution/lifecycle 就是这四件的名字；不做的死法：ADD-only 半年后每用户几千条、60% 近重复/过时 → top-k 被垃圾挤占 → **precision 静默塌方**：

1. **语义去重**（= entity resolution）：新 fact embedding 对本用户全库内积（亚毫秒，异步路径上几十 ms 都是奢侈），超阈值合并/同簇；
2. **矛盾裁决**（去重的邪恶双胞胎：语义相关但内容互斥——"在 Meta 工作"vs"被 Meta 裁了"；高相似 ≠ 矛盾，"喜欢咖啡/喜欢深烘"是细化，需 NLI 级判断——模型归邻组、管道归本组）：写时 ADD-only + 时间元数据 + 矛盾标记，读时 temporal reasoning 裁（新者胜但旧的保留为历史事实）；T6 物化：高频矛盾对批量预载存结论 = 读时裁决的物化视图；
3. **衰减重打分 + quota 压缩**：score = 重要性 × 置信度 × 时间衰减，影响检索排序与压缩优先级而非删除；撞 quota 时低分碎 fact 合并成 summary（降精度不清零）；
4. **重放通道**（backfill）：抽取模型升级后从 raw text 重炼——留原文真身的意义在这一刻兑现。

三件共同骨架：**新旧知识关系的三种 case——重复（合并）、互斥（标记 + 读时裁）、陈旧（降权 + 压缩），一条 T6 批管道顺序跑完。**

边界（机制 vs 策略）：本组管机制（管道、schema、调度、回放——怎么做）；算法组管策略参数（什么算矛盾、衰减系数、重要性定义——做成什么样）。类比：推荐 infra 建特征商店和样本流，不定 ranking 目标函数。**把腿二整个划成"业务逻辑"会被 JD 原文钉——砍掉腿二这个组就退化成纯存储组，而纯存储恰是隔壁的地盘。**

8.3 成本结构：三层漏斗，GPU 缩到塔尖

业界实锤（本次搜证）：抽取类任务正在从小 LLM 退回 CPU encoder——GLiNER 族（<500M encoder，zero-shot NER/IE 追平 LLM，标准 CPU 高效运行，PII 领域大规模落地）；蒸馏路线（LLM teacher 标注 → BERT student，信号自举环的历史结果就是蒸馏训练集，管道自己喂自己）；NLI 矛盾判有 CPU 档（TinyBERT cross-encoder，CPU ~50ms、差 GPU 仅 3%）。CPU 的边界：7B+ 生成式推理撑不住。

- 1. CPU encoder（80-90% 的活）：embedding、去重内积、GLiNER 式显式 fact 抽取、NLI 初筛（embedding 相似 + 实体匹配先筛 candidate pairs）——跑在自己 4000 台刀片的空闲核上，谷时 CPU 本来闲着，数据本地零网络；
- 2. GPU 小模型（塔腰）：过筛难 case——隐式事实推断、多轮 summary 合成；
- 3. GPU 大模型（塔尖，采样）：质检 + 蒸馏 teacher 标注。

让免费的 CPU 把贵的 GPU 挡在后面——推荐 cascade 的原样平移。三个既有杠杆叠加：增量触发（只处理有新 fact 的用户）、错峰吃谷时算力、模型分级。

三本账（终版）：

账本	内容	记账人
T4 生成 GPU	2.8M turn/s × ~1 GPU-秒 → 百万 GPU 级，全系统大头	推理组的账 + per-user quota 产品定（本组只上报消耗，计量表不是收费站）
T5/T6 炼化	CPU 化后近零：CPU 批作业为主 + GPU 塔尖采样	本组配额
CPU/RAM	4000 台热池、分片、批作业	本组一等资源

杠杆论（面试金句）："T4 的钱我们省不了，但我们决定 T4 每一块钱花得值不值——context 组装的质量（对的 memory、最少的 token）直接决定那百万 GPU 产出的答案质量和 prefill 长度。我们是全系统最贵账单的杠杆，不是它的付款人。"

9. 资源版图与预算谈判

9.1 热层四租户台账（终版）

租户	稳态	Quota/buffer
Session state	~20TB	50TB
Fact + embedding（含版本并存 2-3×）	~100TB	200-300TB
In-flight context	~10TB	50TB（cap 口径）
小计（T0-T4 状态类）	~130TB	~400TB（硬底）
检索热索引（对话倒排/向量热部，~1% LRU）	—	剩余 ~1.6PB，主租户

结论：T0-T4 全家连 buffer 顶格 20% 热池；2PB 的主体始终是检索索引热部——预算的真正战场在索引准入策略（哪些用户/chunk 配进 RAM）。

9.2 2PB 的身份：申请书，不是赠予

2PB 是自己拍的锚（1% 热数据经典 working set 点位）。谈判框架：

- 自下而上：硬底 ~400TB（状态类租户；撤退线 = 索引全下 SSD，P99 涨几 ms，产品可活体验降档）；弹性段 = 检索索引，每加 1PB 买回 X% RAM 命中率（X 从 trace 仿真出）；
- 自上而下：全公司 RAM 池 ~几百 PB（百万台级 × 256-512GB）；参照系——推荐（稀疏 embedding table）几十 PB 档最大户、搜索/Ads 各 10PB 档、通用缓存层几十 PB。AI agent 业务线合理落点 1-5PB 档（占全池 ~1%，与 1B DAU 战略产品相称）；
- 成本结构反转（高一档的答案）："RAM 对这条业务线不是稀缺资源——稀缺的是 GPU；热层 1-5PB 相对推理集群成本是个位数百分比，所以设计原则是用便宜的 RAM 换贵的 GPU（多缓存少重算、context 复用），这个替换方向本身就是预算论证。"

本组资源身份证：RAM/SSD/HDD 是一等资源（自己规划申请优化）；GPU/显存是租的（推理侧接口 + quota，本组调用方，显存怎么切/KV cache 怎么管不越界）。生存策略：用自己的一等资源（RAM 阔绰、CPU 闲核）对冲租来资源（GPU）的紧张。

9.3 全球 GPU 参照系（2026 年中口径）

全球部署/售出 ~1600-2000 万张 H100 等效；按 ~100GB/卡折算 **全球 HBM $\approx 1.5-2$ EB**。分家：Google ~500 万卡（~25% 全球算力）→ HBM ~400-500PB；OpenAI ~170 万卡（租 Microsoft/Oracle/CoreWeave 为主）→ ~150PB；Meta ~百万卡级 → ~100PB；xAI Colossus 55.5 万卡（~180 亿美元、2GW 站点）→ ~50-80PB，且利用率仅 ~11%（Meta/Google 43-46%——买得起和用得满是两回事）。**一线公司显存池 100-500PB，比本组 2PB RAM 申请大两个数量级——"GPU 稀缺"稀缺的不是它的存储，是算力 + 采购 + 功耗；本组要的资源在公司版图里是零头，谈判难度低，这本身是架构优点。**

10. 方法论沉淀（本次推演的交锋记录）

这两场推演中反复出现、可直接迁移到面试现场的纪律：

- 锚定纪律**：一个 master 锚，所有派生数字与它对冲；改锚全局同缩，新旧不混用。三处滑坡（200 万并发侧估、0.5KB 口径漂移、270PB 单位滑坡）全靠对冲抓回。报数姿势：报区间 + 报敏感变量（"TB-10TB，取决于活跃系数，需要数据定"）比报精确数更 staff。
- 分母检查**：任何"率"先问分母是什么（per access vs per segment，命中率差一个量级）。
- 钝赢精巧（三连）**：TTL/LRU 胜 session 感知逐出；ADD-only + 读时裁决胜写时 reconcile 状态机；trace replay 胜排队论解析解。精巧的方案往往在耦合两条本来解耦的路径。
- 机制 vs 策略**：本组管道/schema/调度（怎么做），算法组管判断标准（做成什么样）。系统设计现场把这句边界说出口本身是得分动作。
- 解析证结构，仿真出数字**：Little's Law/共轭/80/20 回答量级和因果方向；trace-driven simulation 回答具体数字 + 分位数 + 参数扫描，挂周期回归作业。
- 业界对齐先行**：自己推完必查业界实际做法（mem0、Discord、Redis 参考架构、GLiNER、adaptive RAG），偏差逐条标注——四次"毙掉模板/过度设计"（ $\wedge$ 控制器、session 感知逐出、composite key、温层再切）都发生在对齐动作之后。
- 决策的三种命运**：消灭（成本 < 判断成本 → 无条件做）、摊销（搭反正要发生的调用的便车 → tool call）、保留（真正稀缺处才建控制器）。
- Little's Law 三用**：并发人数（DAU $\times$ 时长/24）、驻留字节（流入速率 $\times$ 驻留时长）、in-flight 请求（QPS $\times$ 响应时长）——流率积成存量，一个恒等式三种量纲。
- 同一思想在两层各长一次**：delta read（数据层）与 prefix caching（推理层）；倒排（能重建但必存）与 embedding（物化索引）。认出同构，表述就省一半。
- Depth-to-purpose（上一场立的总纲）**：估算尺度、优化取舍、整题深度、跨题型认知资源分配——四次校准同一技能；"这题在面试所需深度上闭合，每个子系统知道往下还有什么、被问能接、不主动钻"这句话本身可以在面试里说。

11. 面试速查卡

- 开场 spine**：双写（在线 KV + 数仓）→ 离线炼化 → 回灌在线检索；critical path 只读不写。
- 不存 KV 一句话**：可重算 + 贵 10^5 倍 + model-bound + 单 session 复用；存 raw text（source of truth）+ context 快照 log（不可重放，训练数据主体）。
- 检索一句话**：v0 lexical + rewriting（同人词汇分布 + BEIR + 零训练）；dense 是 signal building 做成后的 v1；信号五级级联 + session reformulation as reward；per-user 小库 brute-force 内积，不用 ANN。
- T0 一句话**：有序 KV/LSM、(userid, sessionid, bucket)、2h 滑动 TTL + jitter + volatile-lru + 50TB quota、miss 1.5% 段首集中由生活节律锁死、容量论证三重闭合、参数由 trace replay 定。
- T1 一句话**：热层 fact 常注入（决策消灭），冷层 tool call（决策摊销），pre-router 只做 prefetch。
- T5/T6 一句话**：per-turn 异步四件写；T6 = 炼化 + compaction 两条腿（LSM 类比）；新旧知识三 case = 合并/标记读裁/降权压缩；成本 = CPU encoder 漏斗 + GPU 塔尖。
- 资源一句话**：RAM 大户、GPU 租客、T4 杠杆——"我们决定最贵账单每一块钱花得值不值。"
- 欠账清单（下次续）**：T6 具体机制深挖（矛盾裁决物化/衰减函数/金字塔触发）、multimodal fusion、routing 深挖、mem0/memOS 源码扫读、event tracking pipeline mock 未答、reconciliation mock 欠着。

12. 贯穿全篇的七个不变量（升维层）

整个系统看似上百个决策，抽象掉名词后只有七个思想在反复实例化。认出同构，是把"背了一堆结论"变成"看穿一类系统"的那一步——面试里能在两个看似无关的子问题之间点破"这是同一件事"，就是评审印象从 senior 跳到 staff 的瞬间。

12.1 不变量一：L = λW（流率积成存量）

一个恒等式，三种量纲，四次出场：

出场	λ（流率）	W（驻留）	L（存量）
并发用户	11.6K session/s	4h	1.67 亿并发 session
热池字节	5.5 GB/s（480TB/天）	~4.5h（session 生命周期 + TTL 尾巴）	~90TB raw → 压后 30-45TB
In-flight 请求	2.8M prompt/s	~10s 响应（60s cap）	8TB 典型 / 50TB cap
推理侧 KV 显存	并发 turn 到达率	prefill+decode 时长	并发 × context 长度 × per-token KV（别组的账，同一定律）

它的深层价值是**消除一类幻觉**："一天 480TB 都要在热存里"这个直觉错误，本质是把 λ 的日积分当成了 L。任何时候看到大数，先问：这是流过系统的量（flow），还是驻留在系统里的量（stock）？stock 永远 = flow × 停留时间。管子的粗细由流速 × 管长决定，与一天流过多少水无关。

12.2 不变量二：可重算三分法（数据的身份由重算经济学决定）

可重算且便宜 → 不存；可重算但贵且高频复用/结构性依赖预计算 → 存为 cache/index；不可重算 → 必须存为 log。

一张判据表裁掉全部"存不存"争论：

对象	可重算？	重算成本	复用模式	判决	身份
Raw text	否（世界的输入）	—	一切的源头	必存	source of truth
Context 快照	否（时变系统状态的函数）	不可重放	训练 memory 系统	必存	log
Embedding	是	便宜（~1PB）	一条向量服务所有未来查询 + 检索要求索引先于查询	存	物化索引
倒排	是	便宜	同上	存	物化索引
KV cache	是	prefill 便宜（compute-bound）	只服务本 session + model-bound	不落仓	分钟级 serving cache
QKV 当训练数据	是	边际信息量 = 0	backprop 插不进	不存	幻觉
Model	是（训练产物）	—	registry 几份	存指针	artifact

深层一句：**数据的价值不在"有没有信息"，在"信息是否已经在别处、以及取回的成本结构"**。activation 是模型对文本的反应，不是世界给模型的信息；raw text 是充分统计量。

12.3 不变量三：钝赢精巧（解耦的胜利）

四场对决，钝件四连胜，胜因全部是同一个——**精巧方案在耦合两条本来独立的路径**：

精巧方案	钝方案	精巧方案偷偷耦合了什么
session 感知逐出（"炼化完才踢"）	2h 滑动 TTL + volatile-lru	把 RAM 逐出与炼化管道耦合——炼化输入走数仓，本不依赖 RAM 那份
写时 reconcile 状态机（UPDATE/DELETE）	ADD-only + 时间元数据 + 读时裁决	把写入延迟与语义判断耦合——判断可以推迟到读，写路径必须快

精巧方案	钝方案	精巧方案偷偷耦合了什么
composite key (taskid 进 key)	(userid, sessionid) + 有界读 + 应用层过滤	把存储布局与一个尚未发生的计算 (分类) 耦合——循环依赖
per-prompt λ 控制器	热层常注入 + tool call	把决策与独立组件耦合——决策可以消灭 (成本倒挂) 或摊销 (搭便车)

钝件的共同美德：每个组件只对一个简单条件负责（时间、容量、追加、复用），组件之间通过“可回载/可重放/可重建”解耦——**任何一层丢了都能从下一层重建，所以任何一层都可以激进**。这就是“能在生产里跑十年”的设计与“评审时漂亮”的设计的分界。

12.4 不变量四：共轭对（每个收益都有它的价格标签）

系统里没有免费的旋钮，每个旋钮连着一对共轭量：

- **miss $\leftrightarrow$ 内存**：均匀访问 = 零 miss 但全员常驻 110TB；突发 = 3-4 次段首 miss 换 20TB。用户的睡觉时间就是内存预算，miss 是省钱的收据；
- **压缩比 $\leftrightarrow$ 访问模式**：大 block 批压缩 8:1 换来只能顺序扫；record 级 2-3:1 换来随机读写。160TB vs 60TB 的差价就是随机访问的标价；
- **token 省 $\leftrightarrow$ 索引费**：agentic search 花 token 省索引，embedding 花索引省 token——Claude Code 与 Cursor 的路线分歧就是这对共轭在两个 QPS 区间的不同解；
- **写延迟 $\leftrightarrow$ 一致性债**：T5 append 逃逸写延迟，把去重/矛盾/精简欠成债，T6 compaction 还——LSM 的灵魂；
- **网络税 $\leftrightarrow$ 弹性**：计算随数据免读税但锁死两层伸缩节奏；无状态 serving 交 12GB/s 小税换独立扩缩。

面试的高分姿势不是“选对了哪边”，是**把共轭对摆上桌面、指出自己的刀划在哪、为什么划在那**。“TTL 2h $\rightarrow$ 8h 买 1 个点命中率、内存翻 4 倍，不值”——一句话里有整个框架。

12.5 不变量五：漏斗（让免费的挡在贵的前面）

同一个 cascade 思想在四处独立生长：

1. **检索**：lexical（免费） $\rightarrow$ 向量内积（便宜） $\rightarrow$ LLM rerank（贵） $\rightarrow$ 生成（最贵）；
2. **T6 判断**：哈希去重（免费） $\rightarrow$ embedding 相似 + 实体匹配筛 candidate（便宜 CPU） $\rightarrow$ NLI 小模型（CPU 50ms） $\rightarrow$ LLM 判难例（GPU） $\rightarrow$ 大模型抽检（GPU 塔尖）；
3. **信号生产**：执行反馈（免费、量大、当主食） $\rightarrow$ utilization 采样精标 $\rightarrow$ 金标校准（最贵、只当裁判）；
4. **决策**：常注入（决策消灭） $\rightarrow$ tool call（决策摊销） $\rightarrow$ pre-router prefetch（可选加速器）。

推荐系统 multi-stage ranking 的原样平移——粗排挡在精排前面，精排挡在重排前面。**漏斗的设计准则只有一条：每一级的职责是让下一级更贵的资源只见到值得见的候选。**

12.6 不变量六：同一思想在相邻两层各长一次

- **delta read（数据层） $\leftrightarrow$ prefix caching（推理层）**：都是“同一会话的公共前缀已在近端，每轮只传/只算增量”。前者砍 KV $\rightarrow$ serving 的 300GB/s 到 12，后者砍 serving $\rightarrow$ GPU 的 840GB/s 到 30。协作接口一句话：session 亲和路由 + 增量可辨识 (seqno) ；
- **LRU（缓存层） $\leftrightarrow$ 回访概率估计（统计层）**：LRU 不是“策略”，是对回访间隔分布的在线非参估计——所以不需要离线解那个重尾 CDF，线上自适应逼近拐点；
- **倒排（词项空间） $\leftrightarrow$ embedding（语义空间）**：都是“能从原文重建但没人不存”的物化索引，hybrid 检索就是两个索引空间的 rank fusion；
- **trace replay（容量） $\leftrightarrow$ 反事实 ablation（信号）**：都是“解析解难就直接在历史数据上重放另一个世界”。

认出“这是 X 在 Y 层的镜像”，表述立省一半，且暴露的是对机制而非名词的理解。

12.7 不变量七：边界即设计（说清“不做什么”是结论不是免责）

全篇最常被低估的动作是划界，但每条边界都改变了设计本身：

- **TAO/UI 划出去** $\rightarrow$ inference 侧只剩 raw text + log，三分法才推得动；
- **ANN/全局检索划出去** $\rightarrow$ per-user 小库 brute-force 成为终解，5000 维之类的过度设计失去土壤；
- **生成 model 选择、全局 GPU 池划出去** $\rightarrow$ λ 控制器解散，T1 坍缩成两层；
- **机制 vs 策略切开** $\rightarrow$ 腿二留在界内但判断模型归算法组——组的存在理由（不是纯存储、不是算法）恰好由这条缝定义；

- **RAM 一等资源 vs GPU 租用** → "用便宜的 RAM 换贵的 GPU"从一句口号变成预算论证本身。

系统设计面试的最后一道分水岭：**junior 把边界当"没时间讲"**，**senior 把边界当"知道但不展开"**，**staff 把边界当设计结论说出口**——"这题在面试所需深度上闭合，每个子系统知道往下还有什么、被问能接、不主动钻"。

13. 数字派生树（每个数从锚到叶的完整血统）

面试中报任何数字，都应该能当场沿这棵树回溯到锚。反过来，锚一改，整棵树按叶子上的倍率同步刷新——这就是"改锚是全局事件"的可视化。

```
锚：1B DAU × 240 条/天 × 2KB/条
├── 流量支
│   ├── 480 TB/天 raw (×压缩 5:1 → 数仓 96TB/天 ≈ 60-100TB 档)
│   ├── 175 PB/年 raw → 35 PB/年 压缩 → 冷层 EC 1.4× ≈ 49 PB
│   ├── 2.8M msg/s 均值 (÷86400) → 峰值 ~8M/s (×3)
│   └── 读带宽 = 2.8M × context 尺寸
│       ├── ×100KB (session state 全量读) = 300 GB/s → delta read 后 12 GB/s
│       └── ×300KB (完整组装 context → GPU) = 840 GB/s → prefix cache 后 ~30 GB/s
├── 并发支 (Little's Law)
│   ├── 并发 session = 1B × 4h/24h = 1.67 亿 (敏感变量：人均时长；30min 档 → 2000 万)
│   ├── session 到达率 = 1B/86400 ≈ 11.6K/s (= miss 率的分子来源)
│   └── in-flight = 2.8M × 响应时长 (10s → 2800 万；60s cap → 1.68 亿)
├── 存量支 (= 流量 × 驻留)
│   ├── 单 session state = 240 × 2KB = 480KB raw → 压后 ~110KB (含 centroids/元数据)
│   ├── session 热池稳态 = 5.5GB/s × ~4.5h ≈ 90TB raw → 30-45TB (quota 50TB)
│   ├── fact 层 = 300KB/人 × 1B = 300TB 全量；热部 = 1.67 亿 × 300KB ≈ 50TB (quota 含版本 buffer 200-300TB)
│   ├── in-flight context = in-flight 数 × 300KB (8TB 典型 / 50TB cap)
│   └── 热池总账：状态类 ~130TB 稳态 / 400TB 硬底；检索索引占剩余 ~1.6PB (主租户)
├── Miss 支
│   ├── miss/s ≈ 间隔>2h 的段到达率 ≈ 40K/s (3.5 段/人/天，生活节律锁死)
│   ├── miss 率 = 3.5/240 ≈ 1.5% (区间 0.4%-2.5%，极端上限 8%)
│   └── 回载流量 = 40K × 100KB = 4 GB/s (持久层零头)
├── 机器支
│   ├── 热池 = 2PB ÷ 500GB 可用/台 = 4000 台 (session state 寄生 13GB/台 = 2.6%)
│   ├── KV QPS 口径 = 2.8M ÷ 10 万 ops = 28 台均值 / 80 台峰值 (带宽驱动则 ~100 台，delta read 后回落)
│   └── 推理侧 = 2.8M × ~1 GPU-秒/turn → 百万 GPU 量级 (别组的账，参照系)
└── 炼化支
    ├── T5 抽取 = 2.8M 次/s (per-turn，本组账单第一行 → CPU encoder 化后近零 GPU)
    ├── T6 批 = ~10^9 用户 × 日均几次判 ≈ 1-2 万次/s (错峰谷时)
    └── per-user T6 单轮 ≈ 秒~十秒级 (内积 μs + NLI 百 ms + 触发式 summary 秒级)
```

三条使用纪律：报数先报锚 ("按 240 条/天的锚")；报区间带敏感变量 ("TB-10TB，取决于活跃系数")；被质疑时沿树回溯而不是重新拍。

14. 十五分钟口述骨架（面试现场版）

以下是把全篇压成面试现场语流的骨架——每段给的是"说出来的话"的密度和顺序，括号内是伴随动作。真实现场按面试官打断随时下钻，但主线不变。

[0-2 分钟] 题目复述与边界宣告。"我理解要设计的是 agent 的记忆子系统：对上暴露 ingest 和 retrieve 两个接口，对下租存储引擎和 GPU 推理，邻居是编排、推理服务和模型训练三个系统——这三块我会点到接口为止。锚定一下规模：1B 用户、每人每天 240 条、每条 2KB——总量 480TB/天、2.8M QPS，后面所有数字从这组锚派生。"（白板左上角写锚，右上角画三明治边界。）

[2-5 分钟] 数据面与"存什么"。"先回答存什么。判据是重算经济学：不可重算的必须存——raw text 和推理时刻的 context 快照，后者是时变状态的函数、事后重放不出来，也是训练检索排序的核心数据，等价于 ads 的特征快照。可重算但结构上要求预计算的存成索引——embedding 和倒排。可重算又贵又只服务单 session 的不存——KV cache 是原文十万倍、还随模型作废，它只配当推理引擎的分钟级 cache。

分层：热 RAM 放活跃状态和热索引，温 SSD 三个月原文 5:1 压缩约 9PB，冷 HDD 全量加 EC 约 50PB/年。压缩比是分层属性——大 block 换高压缩、小 record 换随机访问。"

[5-8 分钟] 读路径与检索。"一个 prompt 的旅程：读 session state（有序 KV、userid+sessionid 分片、2 小时滑动 TTL 加 LRU，稳态几十 TB、命中 98% 以上、miss 集中在段首由生活节律锁死约 1.5%）；便宜的记忆——profile 和高置信 fact——每轮无条件注入，不做决策，因为判断比注入贵；贵的检索暴露成 tool 由生成模型自己调，决策搭生成的便车。检索本身：per-user 语料才十万级向量，brute-force 内积就是终解，不需要 ANN；v0 用 lexical 加 LLM query 改写——个人记忆场景 query 和语料同人词汇分布，lexical 天然强，BEIR 也证明无域内数据时 BM25 赢 dense。dense 是攒够 utilization 信号后的 v1——这个场景没有天然 click：没有 SERP、corpus per-user、label 靠推断，所以 signal building 本身是这个系统最难的子问题。"

[8-11 分钟] 写侧——这个组的心脏。"写全部异步、critical path 只读不写。T5 逐轮四件：append 状态、双写数仓、落 context 快照、抽 fact——抽取用 CPU encoder（GLiNER 一族追平 LLM 且跑在自己刀片的闲核上），GPU 只兜难例。T6 是 compaction：ADD-only 的库半年就烂——去重、矛盾、陈旧三种新旧关系分别走合并、时间元数据加读时裁决、衰减降权加 quota 压缩。类比 LSM：T5 是 memtable append 逃逸写延迟，把一致性欠成债，T6 还债。遗忘机制上 TTL 管缓存、取代加衰减管记忆——过敏史十年不访问也不能过期。"

[11-13 分钟] 成本结构与资源版图。"三本账：全系统大头是生成——百万 GPU 量级，记推理组账上，我们只上报消耗；本组配额里的炼化 CPU 化后近零；RAM 是本组一等资源，热层申请 2PB——硬底 400TB 是状态类，弹性段是检索索引，具体数字用 trace 仿真的命中率-成本曲线去谈。设计哲学一句话：用便宜的 RAM 换贵的 GPU——缓存命中一次就是少一次推理调用；以及我们决定最贵账单每一块钱花得值不值——context 质量直接决定那百万 GPU 的产出。"

[13-15 分钟] 收尾与主动留钩。"这套设计每个组件都有零训练的 v0，日志从第一天为 v1 铺路——系统先跑起来，信号自己长出来。我没展开的：矛盾裁决的物化、summary 金字塔的触发、multimodal 融合、GDPR 级联删除的全链路——每个被问到都能往下走一层。"（留钩的每一项都真的备了一层。）

被打断时的三个万能动作：报数先报锚再报区间和敏感变量；被质疑先找不变量对撞而不是改口；被引向界外先给接口一句话再问"要往那边展开吗"。

附录 A：关键结论的推演过程详记

正文给的是结论的静态形态；本附录还原每个结论是怎么被逼出来的——初始假设、量级检验、反例、修正、收敛。推演路径本身是可复用的资产：同类问题换个皮，走的还是这几条路。

A.1 "存什么"的推演：从"全存"收敛到三分法

起点假设：inference 相关的数据全存——原始对话、原始 QKV、对应的 model，至少存一个月，将来都是训练数据。

第一次量级检验立刻筛掉 QKV。先算单位成本：Q 不需要存（当前 token 现算现用，attention 里只有 K/V 被缓存）；70B 级模型每 token 的 KV = $2 \times 80 \text{ 层} \times 8 \text{ 个 KV head} \times 128 \text{ 维} \times 2 \text{ 字节} \approx 320\text{KB}$ ，而 raw text 每 token $\approx 4 \text{ 字节}$ —— **10^5 倍**。代入锚： $1.2 \times 10^{13} \text{ token/天} \times 320\text{KB} \approx 4 \text{ EB/天}$ ，一个月 120 EB，超过一线公司整个数仓。只存最后一层（32KB/token）也要 380 PB/天 $\approx$ 压缩原文两年的量。存储账直接毙掉。

但量级账没有回答"值不值"，只回答了"付不付得起"——于是追问：假如存得起，最后一层的 QKV 是不是好的训练数据？这一问把推演逼到信息论层面：activation 是 $f(\text{raw text}, \text{model weights})$ 的**确定性函数**，给定原文和权重随时可精确重算，相对 raw text 的边际信息量为零——它不是数据，是缓存的计算。训练数据的价值在于含有模型没有的信息（用户说了什么、什么回答被采纳），这些全在 raw text 里。且当训练数据用也省不掉重算：backprop 需要下面所有层的 activation 在 forward 时重建，存的最后一层插不进流程。**"最后一层有信息"这个直觉的合理内核由两个正规出口承接：**要软标签 $\rightarrow$ 存 top-k logits（ $\sim 100\text{B/token}$ ，对没有 teacher 权重的 student 是真信息，且只对采样子集存）；要语义 $\rightarrow$ 存 pooling 后的 embedding（1~4KB/chunk）。

第二次追问反向进攻："推理用到的 feature 总得存吧？而 embedding 反而不用存——model 本身就能从 token 生成它。"这一对问题恰好互为镜像，把判据逼成完整的三分法：

可重算且便宜 $\rightarrow$ 不存；可重算但贵、且高频复用/结构性依赖预计算 $\rightarrow$ 存（身份是 cache/index）；不可重算 $\rightarrow$ 必须存（身份是 log）。

- 推理时刻的 context 快照落在第三类：进 context 的内容是**时变系统状态的函数**（那一刻检索出了哪几条、索引内容每天在变、profile 当时的快照、model version），事后拿 raw text 重放不出来——与 ads ranking 的特征快照（serving 时刻落特征、防 training-serving inconsistency）是同一个模式。它必须升格为"完整快照 + version"而不只是 query $\rightarrow$ 召回列表。

- embedding 落在第二类：边际信息量确实为零（这半句推理成立），但存它的理由从来不是信息量——ANN 检索在数学结构上要求**全库向量先于查询就绪**（每个 query 要和全库比，不可能 query time 现算全库），加上“一条向量服务所有未来查询”的极端复用和 ~1PB 的低价。它的身份是**物化的索引**，与倒排同族：能从原文重建，但没人不存倒排。
- KV 落在第一类的反面教材：可重算 + 贵 10^5 倍 + 复用局限单 session + model-bound（模型一升级全作废）——正确定位是推理引擎的分层 serving cache（HBM→CPU RAM→NVMe，分钟~小时 TTL），不进数仓。

收敛：inference 侧长期存两样——raw text（source of truth, model-agnostic）+ context 快照 log（训练数据主体）；embedding 划归检索块（物化索引）；QKV 不存；model 存 registry 指针。同一个判据，四个对象四个结论。

A.2 检索训练信号的推演：从"没 click 训不了"到信号生产线

起点困境：dense retrieval 的标准配方（双塔 + click 日志）在这个场景缺原料——"没有 click，不好训吧"。

第一轮方案（pseudo-query + LLM-as-judge）经不起检验。从 memory chunk 反推 query 构造训练对，噪音是**结构性的**：生成的 query 太规整，真实 query 破碎、指代不清、上下文依赖，分布 gap 系统性存在；LLM-as-judge 判得了 (q, chunk) 相关性，判不了 q 本身的代表性。冷启动伪标签只能保证 v0 不瞎，撑不起质量宣称。

替代方案"宽召回 + LLM 总结精选"看似绕开训练，实则把账挪了位置。成本检验：把 precision 的活从毫秒级/微美元的检索层挪到百毫秒级/毫美元的 LLM 层，单位成本差 10^3 ；延迟检验：检索→总结→回答两次 LLM 串行。但这个方案有一个场景特有的强支撑：**个人 memory 检索里 query 和 corpus 出自同一个人的词概率分布**——用户找自己说过的话，用词高度一致，vocabulary mismatch 天然弱，lexical 的 recall 端远比通用搜索站得住。

合成不是二选一，是时序分工：宽召回 + LLM 精选做 v0（零训练依赖、一周上线），而它跑起来的那一刻，成本问题自动变成数据来源——LLM 总结时"用了哪几条扔了哪几条"就是真实 query 分布下的 utilization label，比 pseudo-query 干净一档。dense 是拿这些 log 训的 v1，存在理由不是效果，是把宽召回收窄、省回 LLM 层的钱。BEIR 的历史结论（BM25 zero-shot 域外碾压无域内数据的 dense）给这个时序背书。

信号从哪来的完整推演产出五级级联（正文 §3.3 表），其中最关键的一级是独立推出来的：**session 内的下一个 prompt 就是对上一个 query 的评分**——用户重述换词 = 负，深入追问 = 正，换话题 = 无标注。这与 web search 二十年的 query reformulation as implicit feedback 文献完全同构，但两个坑必须当场钉住：归因混杂（继续追问的责任方可能是 query 烂/库里没有/生成没用好——用执行反馈层做条件消歧）；分布污染（下一个 prompt 是用户看到失败回答后的反应，**只能当 reward 不能当 seq2seq 的 label**）。

"per-user corpus 训不了"的反例检验：Google 搜索也 per-user 个性化——但拆开看不构成反例：web search 是共享 corpus + per-user 特征，doc 级信号照样十亿人聚合。真正的同类是 Gmail/Drive personal search，那条文献线的困境与解法恰好完成推演：doc 级点击无法聚合 → 信号参数化到**属性空间**（term match、recency、affinity），跨用户迁移。推论落到本设计：双塔若训，doc 塔不得有 memory-ID 参数，纯内容编码——被逼的约束反而利于迁移，也反向加固了 lexical v0（连模型都被逼进内容空间，lexical 本来就住在那里）。先发优势的精确形态随之清晰：Gmail 有 per-user corpus 上的真 SERP click，但合规墙决定资产在模型与方法论、不在 raw content；对没有这份遗产的玩家，signal building 是护城河的建造现场——这正是该组存在的理由。

query generation 单模型化的推演：（用户问题, history）→ query 一个模型统一内外。训练阶梯 prompt → rejection sampling（只留检索成功的）→ RL on 可验证 reward（Search-R1 路线——reward 从下游结果来，query 分布是策略探索出的真实分布，pseudo-query 的 gap 被绕开）。无监督 LM on history 的角色要切准：它白送的是**表征**（这个人的词表、风格、query likelihood 先验），但 LM 目标 ≠ query 效用目标——对话里根本不出现检索式，好 query 要判别性强的词，而概率高的词恰恰不判别（TF-IDF 存在的原因）。**表征无监督白拿，策略必须付费，币种是 utilization log。**"取代 embedding retrieval"的判定：Claude Code（纯 agentic 迭代搜索）与 Cursor（embedding index）的公开分歧给出分界——花 token 省索引 vs 花索引省 token；低 QPS 高价值可迭代场景 agentic 赢，1B 用户每轮检索、latency 在 critical path 上，query gen 是检索层前端而非替代。账在 QPS，不在效果。

A.3 T0 的推演：一张 session 表如何被逼到只剩钝件

Schema 的推演。初始方向是把 taskid 编进 composite key 优化"按 task 读"——被一个循环依赖毙掉：查询时 taskid 尚未算出（分类本身依赖读出来的 context）。于是 key 收敛为 (userid, sessionid[, time_bucket])、时间降序、**有界读 + 应用层 task 过滤**（N 百量级，app-side filter 零成本）；time_bucket 借 Discord 的做法防超长会话打爆单分区。task 聚类模型划归算法组，本组只签系统契约：query embedding 对 K 个 centroid 内积（K 个位数、微秒级）、置信度低于阈值 sticky 沿用上一轮 taskid（省略式短 prompt 恰恰出现在没切话题时——对话连续性先验白用）、开新类 O(1)。

容量的推演经历了三次校准。第一次：拍"200 万并发峰值 1000 万"侧算出 10TB——与 master 锚对撞失败（凑 2.8M QPS 需每人每秒 1.4 条，荒谬），恒等式裁决并发 = 1.67 亿（1B × 4h/24h）。第二次：口径滑坡（0.5KB/条溜进来替换了 2KB 锚）被抓回，单 session state 定格 ~110KB。第三次：混淆流量与存量——"一天 480TB 怎么只存 50TB"。解铃的是 Little's Law 换量纲再用一次：存量 = 流入速率 × 驻留时长 = 5.5GB/s × ~16000s ≈ 90TB raw → 压后 30-45TB，与并发口径（1.67 亿 × 110KB ≈ 18TB）咬合。热池是直通缓冲区不是蒸馏器：每条数据只泡自己 session 的生命周期，"全天数据同时在 RAM"从未发生；160TB（全天随机访问版）是反事实上界，其与批处理版 60TB 的 2.7× 差价就是"访问模式决定压缩比"的价格标签。

逐出策略的推演是一场"精巧 vs 钝"的完整攻防。最初的精巧方案：session 结束 + 炼化完成才逐出（防数据未消费就出内存）。第一处裂缝：真实使用是碎片化的（日均 4h 切成早/午/晚多段），按"session 结束"逐出会让晚间回访 miss——修补为跨段回载 + LRU 自然驻留。第二处裂缝是致命的：查业界（Discord/Messenger/AWS 参考架构）发现**没人写 session 感知的逐出逻辑**——持久 LSM 当主存（最新数据天然在 memtable，SSD 点查毫秒级够 serving），前面通用 TTL cache，逐出交给 LRU；"炼化完成"这个条件耦合了两条本来解耦的路径（炼化输入走数仓/log，根本不依赖 RAM 里那份）。**最初被反对的"2h TTL 上限"方案全面翻案**：数值正中业界主流档（30min-2h），语义之争（固定 vs 滑动）也是稻草人——chat 负载每条消息即一次写、写时重设过期，"固定 2h"自动等价"最后活动 + 2h"。

Miss 率的推演暴露了一个分母陷阱。外部 AI 给出的命中率表（连续使用 90%、四段使用 25%、极端碎片 ~0%）定性排序对、定量全错——分母用了 per segment（只数段首那次加载），而命中率的定义是 per access；段内每分钟一条消息、间隔远小于 TTL，239/240 稳稳命中。重切分母后全部场景 98%+，且"~0%"在锚下物理不可能（240 条两两隔 2h 需 480 小时）。真实的 miss 结构：滑动 TTL 下 miss 只发生在**间隔 > 2h 的段首**，而长间隔的个数由生活节律锁死（隔夜 + 日内深谷 $\approx$ 3-4 次/天） $\rightarrow$ miss 率 ~1.5%，对碎片化钝感。

均匀化思想实验补上最后一块：若 240 条均匀到达（每 6 分钟一条 $\ll$ 2h），滑动 TTL 下永不过期、miss 恒为零——但零 miss 的世界住不起：全员常驻 = $1B \times 110KB = 110TB$ ，爆掉 quota。**miss 和内存是共轭量：均匀 = 零 miss 全驻留，突发 = 3-4 次 miss 换 5 倍内存**——用户的睡觉时间就是内存预算，段首 miss 是省钱的收据，TTL 是在共轭对之间划刀的位置。容量论证至此三重闭合：均值账（Little's Law，20TB < 50TB）、分布账（共轭保证不滑向全驻留）、排序账（80/20 重尾：驻留按人头、命中按流量，LRU 天然把内存花在流量大户上——它本身就是对回访概率的在线非参估计）。

严格解与工程解的分界：给定 30 天访问数据求内存分布，线性期望公式 ($\sum p_i \times s_i$) 是个未声明的近似——驻留时长是**间隔分布的泛函**（滑动 TTL 下的 clump/busy-period 过程： $P(gap > \tau)$ 定簇数、截断均值定簇长；同样日均 4h，均匀撒与三大块驻留差 4 倍），重尾下解析难。工程解干脆不解析：**trace-driven simulation**——30 天真实日志离线重放 TTL+LRU 逻辑，稳态/miss/分位数/参数扫描 (τ 的 miss-内存曲线，CDF 拐点直接画出) 全部直读，挂周期回归作业，行为漂移先于生产报警。**解析证结构，仿真出数字**——直接证明不了的，跑 job 模拟。

A.4 数据放哪、计算放哪：NIC 之争的完整推演

这条线从一个朴素疑问出发："读 session state 是 CPU 读内存，和 NIC 有什么关系？"——五步把架构选择逼到物理层再逼回组织层。

第一步，补全画面的主语。"CPU 读本地内存"缺了"谁的 CPU"：默认架构下读数据的 CPU 在 serving 机、数据在 KV 机，隔着网络——每次读 100KB 走 KV 机 RAM $\rightarrow$ CPU 序列化 $\rightarrow$ NIC $\rightarrow$ 交换机 $\rightarrow$ serving 机 NIC。300GB/s 聚合读带宽 ($2.8M \times 100KB$ ，读字节量是写的 50 倍——"读 heavy 指数度量") 压的是 80 台 KV 机的出向网卡，每台 3.5GB/s，25GbE 打穿。

第二步，拆开两个被合并的决策。"prompt 带 userid，按 userid 直接分配机器"——按 userid **路由**（一行配置，谁都该做） $\neq$ serving 与 KV 分片**同机部署**（网络税的真正开关）。只路由不同机，数据仍要过两边 NIC。

第三步，剥到物理层。判据一条：读数据的 CPU 和存数据的 RAM 是否同一台机器。同机 = 内存总线 (~100GB/s、100ns)；跨机 = 两边 NIC 各过一遍 (~3GB/s、100 μ s)，慢 30 倍。这就是"计算随数据"（data-local/actor 模型，WhatsApp/Orleans 一族）与"无状态 serving + 远程 KV"两个山头的物理分界。

第四步，代价台账。同机方案的四笔账：异构扩缩绑死（CPU 密集编排与 RAM 密集存储同比例扩）、发版滚动重启带状态节点、热点无法独立疏散、故障域合并；最致命的是 re-sharding 节奏锁死——KV 分片长期稳定 vs serving 分钟级弹性。无状态方案交 300GB/s 的税，用 delta read 砍到 12GB/s（serving 侧 session 级软缓存 + 游标增量读；它是 cache 不是副本——本来 T3 组装就要持有这份内存，只是"别扔"；丢了全量读重建，正确性无损）。

第五步，规模翻转了结论的倾向。当口径从"80 台 session 表专属机"换成"4000 台热池刀片"（2PB $\div$ 500GB 可用），计算随数据方案变强一档：50TB session state 摊每台 13GB（2.6%，寄生）；1TB 机型 250GB 分片 + 500GB 计算 + 250GB 系统，比例健康；per-user 全家（state、倒排、向量、fact）按 userid 共置，T0-T3 全程本地；热点疏散靠 virtual shard 切细。而"blade $\rightarrow$ GPU 的 context 传输 840GB/s 省不掉"这条反对被边界切分化解——那是推理接口的账（且 GPU 侧几万台摊下来每台几十 MB/s、prefix caching 再砍到 30GB/s——delta read 在推理层的镜像），不改变"读 session state 这一步 NIC 归零"的结论。

收敛：两个山头都是合法解，选型看组织（存储与编排是否同团队）；面试默认报无状态 + delta read（业界主流），计算随数据当延迟预算极紧时的口袋方案。**路由亲和要；部署亲和看组织。CPU 计算随数据，GPU 计算随批次**（GPU 效率靠跨用户 batching，天然集中，永远不搬）。

A.5 T1 的推演：一个控制器的诞生与解散

起点是一个漂亮的结构识别："每个 prompt 决定查不查、用哪档机器、context 给多少——这有点 pacing 的过程。"映射完全成立：GPU 配额 = daily budget，动作组合 = bid, v/c 贪心 + 对偶变量 λ = 影子价格，PI 控制器盯消耗曲线。理论现成（LP 松弛 $\rightarrow$ λ 阈值 $\rightarrow$ 反馈控制），迁移一气呵成。

三次刹车把它拆了。第一刹：对照 JD——"生成用哪个 model"和"全局 GPU 池分配"两个决策变量越界（model/app 组、平台组的地盘），控制器缩进 memory pipeline 自己的配额。第二刹："难度降了"——其实是难度换位：λ 和档位菜单是机械的，真正的难点集中到 v 的预估（微秒级零 GPU 预测一个百毫秒才知道的结果）。第三刹是致命的："别拿模板套，搜业界"——业界根本不建这个组件：

- 便宜的记忆（profile + 高置信 fact + recent turns）**每轮无条件注入**——几百 token，判断成本 ≥ 注入成本，**决策被消灭**；
- 贵的检索（翻历史/外搜）暴露成 **tool**，生成 LLM 自己决定调不调——判断搭生成调用的便车，边际成本零，**决策被摊销**。"2.8M/s 的判断比生成还贵"是伪问题：不存在独立的判断调用。

pre-router（10ms 小模型）在业界有实锤（Bing Chat 的 search-intent 分类器）但身份不是决策者，是 **prefetch 触发器**——高置信时提前并行发检索把 3-5s 外搜藏进延迟，猜错代价可恢复（白搜/agent 补调）。**Router 管快，tool call 管对**。λ 框架只活下来一根骨头：分层按成本排（免费常注入、贵的按需调），预算控制退化为粗粒度 quota。

这条推演的元教训：模板匹配的诱惑在"结构同构"处最强——pacing 的数学确实同构，但业界用"消灭决策"和"摊销决策"两个更钝的动作让控制器没有存在的必要。结构同构 ≠ 组件同构。

A.6 T5/T6 的推演：从"提取 fact"到"维护一个会烂的库"

T6 的第一版定义（"就是 user level fact 和 embedding 提取"）只覆盖了产出清单。追问"per-turn 能做的 T5 都做了，T6 剩什么"逼出了本体：剩的是所有需要**全局视野或批处理经济性**的活——结构类比一击定型：**T5/T6 = LSM 的 memtable append / compaction**。T5 append 逃逸写延迟（ADD-only、只看眼前的消息对），把一致性和精简欠成债；T6 还债。不还的死法：半年后每用户几千条 fact、60% 近重复或过时，top-k 被垃圾挤占，precision **静默塌方**（没有报错，只有答案变蠢）。

"腿二是业务逻辑不归本组"的边界之争，JD 原文裁决：compression / forgetting / conflict resolution / lifecycle management 就是腿二四件（语义去重、矛盾裁决、衰减重打分 + quota 压缩、重放通道）的名字——砍掉腿二，该组退化为纯存储组，而纯存储恰是隔壁 infra 的地盘。但边界直觉对的一半被重切为**机制 vs 策略**：管道/schema/调度/回放归本组（怎么做），什么算矛盾/衰减系数/重要性定义归算法组（做成什么样）——推荐 infra 建特征商店不定 ranking 目标函数，同构。

三件的统一骨架在逐个定义中浮出：新旧知识关系的三种 case——**重复**（embedding 内积超阈值 → 合并，= entity resolution）、**互斥**（语义相关但内容相反——"在 Meta 工作"vs"被裁了"；高相似 ≠ 矛盾，"喜欢咖啡/喜欢深烘"是细化——写时 ADD-only + 时间元数据 + 矛盾标记，读时 temporal reasoning 裁，热门矛盾对 T6 批量预裁物化）、**陈旧**（score = 重要性 × 置信度 × 时间衰减，影响排序与压缩优先级而非删除；撞 per-user quota 触发碎 fact 合并成 summary——降精度不清零）。一条 T6 批管道顺序跑完。遗忘机制的本体论随之定型：**TTL 管缓存，取代 + 衰减管记忆**——"对花生过敏"十年不访问不能过期。

成本模型的推演经历三次修正。第一版："知识图谱构建，计算内存都不大头，十秒搞定"——per-user 十秒对，但乘上 10^9 用户，LLM 调用量就是之前立过的 GPU 大头账；且"60 秒系统占用"不预付这笔钱——**RAM-秒（占用账，读路径的驻留）与 GPU-秒（做功账，异步错峰的调用）是两本账**，时间不重叠、资源不同类。第二版细看频率：本组账单第一行其实是 T5 per-turn 抽取（频率是 T6 按天批的 200 倍）。第三版"能不能 CPU 做"查业界后定稿：抽取类任务正在从小 LLM 退回 CPU encoder——GLiNER 族（<500M，zero-shot IE 追平 LLM，CPU 高效，PII 领域已落地）、蒸馏 BERT（信号自举环的历史结果就是蒸馏训练集，管道自己喂自己）、TinyBERT 级 NLI（CPU ~50ms 差 GPU 仅 3%）。产出三层漏斗：CPU encoder 干 80-90%（跑在自己刀片的谷时闲核，数据本地零网络）→ GPU 小模型兜难例 → GPU 大模型采样质检 + 产 teacher 标注。**让免费的 CPU 把贵的 GPU 挡在后面**——本组 GPU 账单砍一个数量级。

三本账终版随连环追问定稿：全系统大头当然是 T4 生成（百万 GPU 级）——但记在推理组账上，per-user quota 由产品定，本组唯一接口是消耗上报（**计量表不是收费站**）；T5/T6 记本组配额，CPU 化后近零；CPU/RAM 是本组一等资源。杠杆论收官："T4 的钱我们省不了，但我们决定 T4 每一块钱花得值不值——context 组装的质量直接决定那百万 GPU 的答案质量和 prefill 长度。我们是全系统最贵账单的杠杆，不是它的付款人。"

A.7 预算的推演：2PB 从拍脑袋到谈判框架

三连问完成了升格。"2PB 是全公司的还是这条业务线的？"——回溯出身：它是自己拍的锚（1% 热数据的经典 working set 点位），身份是**申请书不是赠予**。"搜索/ads/feed/游戏都要 hot memory，这条业务线上限多少？"——两头推：自下而上硬底 ~400TB（状态类四租户；撤退线 = 索引全下 SSD，P99 涨几 ms，产品可活体验降档），弹性段是检索索引（每加 1PB 买回 X% RAM 命中率，X 从 trace 仿真出）；自上而下全公司 RAM 池 ~几百 PB，推荐几十 PB 档最大户、搜索/Ads 各 10PB 档，agent 业务线合理落点 **1-5PB**（~1% 全池，与 1B DAU 战略产品相称）。最后一问钉资源身份证："本组主要是 RAM，GPU 显存是别组接口 + quota"——RAM/SSD/HDD 一等资源自己规划，GPU 租用只管调用侧杠杆。成本结构反转是高一档的答案：这条业务线稀缺的是 GPU 不是 RAM（一线公司显存池 100-500PB，比 2PB 申请大两个数量级——"GPU 稀缺"稀缺的是算力/采购/功耗不是它的存储），热层成本相对推理集群是个位数百分比，**设计原则"用便宜的 RAM 换贵的 GPU"本身就是预算论证**。

A.8 推演方法清册

全篇反复生效的推理动作，按触发条件整理：

1. **量级检验先行：**任何"存不存/做不做"先算单位成本 × 规模（QKV 4EB/天、T6 十秒 × 10^9 ）——账不平的方案不需要讨论价值；

2. **找不变量对撞**：估算冲突时找不依赖任何模型的恒等量裁决（总消息量 2.8M/s 裁并发之争）；新估算先撞 master 锚，撞上才算数；改锚全局同缩；
3. **查分母**：任何"率"先问分母（per access vs per segment，差一个量级）；
4. **区分流量与存量**：看到大数先问 flow 还是 stock，Little's Law ($L=\lambda W$) 在三个量纲上通用（并发人数、驻留字节、in-flight 请求）；
5. **拆正交决策**：被合并的选择先拆开（路由亲和 vs 部署亲和；quota 的存在 vs 记账人）；
6. **可重算三分法**：数据的身份由"重算成本 $\times$ 复用模式"定（log/index/cache），不由"有没有信息"定；
7. **精巧方案先过业界检验**：session 感知逐出、composite key、 λ 控制器、温层再切分——四个精巧设计全部被"业界实际怎么做"否掉，钝件（TTL/LRU、ADD-only、tool call、trace replay）四连胜。钝的赢在解耦：精巧的方案往往在耦合两条本来独立的路径；
8. **直觉的内核与载体分离**："最后一层有信息"（对） $\neq$ 存 QKV（错）；"embedding 该带 version"（对） $\neq$ 存 model（错）——反例毙掉载体时，检查内核是否值得换个正规出口安放；
9. **严格解与工程解都摆上桌**：承认解析在近似（线性期望 vs clump 泛函），然后用仿真绕过——解析证结构，仿真出数字；
10. **边界是推演的一部分**：机制 vs 策略、界内 vs 邻组、一等资源 vs 租用——每条边界说出口本身就是设计结论。

附录 B：T6 机制预研（欠账的先行填补）

正文把 T6 立到了骨架层；本附录把四个机制各往下压一层——到"被追问时能画出 schema 和伪代码"的深度，但仍守住机制 vs 策略的界（判断模型的质量归算法组）。

B.1 Fact 的完整 schema（一切机制的载体）

```
fact {
  fact_id, user_id
  text      # 明文本体: "用户住 Jersey City"
  entities[] # 归一化实体 (Jersey City  $\rightarrow$  LOC:xxx)，去重与矛盾检测的键
  t_observed # 事件时间 (用户说这话的时刻)
  t_ingested # 系统时间 (入库时刻) —— 双时间戳，读时裁决靠 t_observed
  confidence # 抽取置信度 (encoder 打分)
  importance # 重要性先验 (健康/身份类高，闲聊类低) —— 策略参数，算法组定
  hit_stats  # 最近 K 次被检索命中/被采纳的时间戳 (衰减与逐出的依据)
  supersedes[] # 指向被本条取代的旧 fact (ADD-only 下的"软 UPDATE")
  contradicts[] # 矛盾标记 (对称边)
  source_span # provenance: 指回 raw text 的 (session, seqno) 区间——重放与 GDPR 都靠它
  emb_version, embedding
}
```

关键设计点两个：**双时间戳**（"3月说的事 11月才入库"时，裁决用事件时间不用系统时间）；**source_span**（每条 fact 可回溯到原文——重放通道、可解释、级联删除三件事共用这一根线）。

B.2 矛盾裁决：检测-标记-读裁-物化四段

检测（T6 批，漏斗）：candidate pair 由"同实体 + embedding 相似度落在中间带"筛出——完全相同是重复（走合并），毫不相关不用判；中间带才可能矛盾。过筛的 pair 进 NLI 小模型（CPU 档），输出 entail/neutral/contradict。"高相似 $\neq$ 矛盾"的典型反例要能随口举：喜欢咖啡/喜欢深烘 = 细化（entail 方向）；在 Meta 工作/被 Meta 裁了 = 状态迁移（contradict + 时间序）。

标记（写侧，ADD-only）：contradict 结果只写对称边 contradicts[]，两条都活着。

读时裁决（默认路径）：检索命中一组含矛盾边的 fact 时，按 t_observed 排序——新者为当前状态，旧者降权但不隐藏（"曾在 Meta 工作"仍是有效历史事实，对"帮我改简历"这类查询恰恰是关键信息）。裁决逻辑放读时的代价是每次检索重复付一小笔判断，换来的是写路径零阻塞 + 裁决策略可随时升级（改读侧代码即可，不用动存量数据）。

物化（T6 优化，可选）：hit_stats 显示某矛盾对被高频检索时，批量预裁一次、把结论（current_state 指针）存下——读时裁决的物化视图。触发条件是频率不是存在：低频矛盾对永远读裁，不值得物化。

B.3 衰减与压缩：一个分数、两个动作

分数形式（具体系数归算法组，形式归本组，因为它决定存什么字段）：

$\text{score} = \text{importance} \times \text{confidence} \times \text{recency_decay}(\Delta t)$, $\text{recency_decay} = \exp(-\lambda_{\text{type}} \cdot \Delta t_{\text{since_last_confirm}})$

要点两个： **λ 按 fact 类型分档**——身份/健康类 $\lambda \approx 0$ （十年不衰），偏好类中速，事务类（“最近在准备面试”）快衰； **Δt 从“最后确认”起算而非“首次观测”**——每次被检索采纳或被用户重申都刷新时钟，高频有用的 fact 自动长寿。这两点合起来实现“过敏史不过期、旧面试话题自然沉底”，不需要任何显式 TTL。

两个下游动作：检索排序（score 作为 rank fusion 的一路信号）；quota 压缩（撞 per-user 上限时，取 score 最低的一簇同主题碎 fact → 一次 LLM 调用合并成 summary fact，supersedes 指向全体成员——**降精度不清零，且可从 source_span 重放恢复**）。

B.4 Summary 金字塔：触发式而非日历式

层级：turn（T5 逐轮抽 fact，不产 summary）→ session summary（session 闲置超时后触发，是 T5 抽取的上下文输入——自耗环）→ 主题/时期 summary（仅在 quota 压缩或检索端发现“同主题碎片过多”时触发）。**反对日历式（每日/每月定时全量）**的理由与增量触发同源：绝大多数用户绝大多数日子没有值得再炼的新结构，定时全量是把 GPU 烧在空转上。金字塔的存在理由自上而下只有一个：**token budget**——同样的记忆量，用更少的 token 送进 context（显存约束的影子在存储层的投射）。

B.5 GDPR 级联删除：source_span 是那根线

forget(user_id, scope) 触发的链条：raw text（数仓 + 温层）→ session state（TTL 自然清 + 主动清）→ fact（按 source_span 反查所有从被删原文炼出的条目——包括 summary fact 的传递闭包）→ embedding/倒排（索引条目）→ context 快照 log（含该内容的快照）。两个工程要点：**删除是异步批 + 墓碑标记先行**（读路径立即看不见，物理清理随 compaction 走——又是 LSM 的形状）；**传递闭包靠 supersedes/source_span 图**——这就是 B.1 里坚持每条 fact 带血统的原因，删除合规是 day-1 的 schema 约束，不是事后补丁。

附录 C：高频追问弹药库（一问一段，现场直取）

Q：为什么不用一个大 LLM 每轮把用户全部历史读一遍，还要什么检索？ token budget 和钱。1MB/用户/20 天的历史 ≈ 25 万 token，每轮全读 = prefill 成本 $\times$ 百倍且超上下文窗口；检索的存在理由就是“用 1% 的 token 传递 99% 相关的记忆”。长上下文模型缓解但不消灭这个约束——成本随长度线性涨，而相关信息密度不涨。

Q：向量数据库选型？Milvus/Pinecone/pgvector？ 这个场景不需要向量数据库——per-user 万级向量 brute-force 内积毫秒级，向量作为普通字段存在 KV 里、随分片共置。向量数据库解决的是“全局十亿级库上的 ANN”，那是推荐/全局搜索的问题。选型题先问规模，再问产品名。

Q：多设备/多端并发写同一用户怎么办？ session 天然按 (user, session) 分 key，两端是两个 session，不冲突；fact 层 ADD-only 天然并发友好（无 in-place update 就无写写冲突），去重与矛盾在 T6 批处理时统一收敛——append-only 的又一红利。

Q：机器挂了丢什么？ 分层回答：session state 副本级持久，丢失窗口毫秒级最多未条（用户重发）；fact/索引有数仓真身可重放；serving 侧软缓存丢了退化为全量读。设计准则：每一层都能从下一层重建，所以每一层都可以激进。

Q：memory 被投毒怎么办（用户或攻击者注入假事实）？ 界内答机制：fact 带 confidence + source_span，低置信不进热注入层；异常检测（单位时间 fact 生成速率、与历史画像的语义距离）是 T6 的一路批作业；仲裁策略归安全/算法组。界外一句：prompt injection 防御在生成侧，不在存储侧。

Q：多模态怎么进来？ 骨架不变，两处扩展：原始媒体存对象存储、KV 只存 link（T0 就定过）；行为信号/图像过各自 encoder 出 embedding 进同一套 fact/索引框架——“multimodal fusion”的 infra 形态就是多一路 encoder 和多一个向量空间的 rank fusion，难点（对齐质量）在模型侧。

Q：怎么评测这个系统好不好？ 分层指标：检索层 recall@k/utilization rate（金标集校准）；memory 质量层——矛盾率、重复率、fact 时效准确率（采样审计）；端到端——“用户重复自我介绍的频率”这类产品级代理指标 + 记忆相关投诉率；成本层——每 turn 的 memory 子系统开销（GPU 调用数、RAM 命中率）。trace replay 做容量回归，金标集做质量回归。

Q：冷启动新用户体验差怎么办？ 没有历史就没有记忆，这是定义不是缺陷；工程上前 N 个 session 走纯 recent-turns 模式 + 显式引导（“告诉我你的偏好”直通 fact 库），fact 库从第一轮就开始积累——系统的价值曲线本来就随使用时长上升，这一点对产品坦白比伪装聪明更稳。

Q：和 ChatGPT memory / mem0 的本质区别是什么？ 形态同构（fact 抽取 + 注入 + 检索），差异在规模带来的相变：1B 用户逼出 per-user 共置分片、CPU encoder 化、错峰批调度、trace 仿真容量——mem0 是单租户库，这里是多租户平台；JD 里 unified platform 那个词的分量就在这。